OMPOI Backend

Developer Implementation Guide

Current Implementation	48%
Working Endpoints	28 of 45
Database Tables	14 tables

Prepared by: Sudhir Giri

Date: January 10, 2026

Version: 1.0

Table of Contents

1. Overview
2. Current Implementation Status
3. Module Implementation Details
4. Pending Implementation Tasks
5. Technical Architecture
6. Security & Infrastructure Setup
7. Production Deployment Configuration

1. Overview

The OMPOI backend uses a modular monolith architecture built with FastAPI and PostgreSQL (NeonDB). This guide provides developers with detailed information about implemented features, pending tasks, and implementation specifications for completing the backend modules.

Implementation Summary

Component	Status
API Endpoints Functional	28 of 45 (62%)
Database Tables Migrated	14 tables
Frontend Integration	65% compatible
Modules Complete	Users, Authentication
Modules Partial	Products, Orders, QC, Admin
Modules Pending	AI Integration, Notifications, Rate Limiting

2. Current Implementation Status

Module Completion Status

Module	Completion	Implementation Status
Core Infrastructure	100%	Fully implemented
Authentication & Users	100%	Fully implemented
Products Module	80%	Core CRUD complete, advanced features pending
Orders Module	70%	Basic workflow complete, payment tracking pending
QC Module	60%	Inspection workflow complete, photo upload pending
Admin Module	30%	Basic structure exists, dashboard pending
AI Integration	0%	Not started - 5 endpoints needed
Notifications	0%	Not started - SMTP integration needed
Rate Limiting	0%	Not started - Redis configuration needed

3. Module Implementation Details

3.1 Users & Authentication Module

Status: 100% Complete

All authentication and user management endpoints are implemented and functional.

Implemented Features:

- User registration with email validation
- JWT-based authentication with bcrypt password hashing
- Role-based access control (buyer, seller, agent, admin)
- User profile management (GET/PUT endpoints)
- Database models: users, user_profiles

Endpoints Available:

- POST /api/v1/auth/register - User registration
- POST /api/v1/auth/login - User authentication
- GET /api/v1/users/profile - Retrieve user profile
- PUT /api/v1/users/profile - Update user profile

Optional Enhancements:

- Password reset endpoint (POST /api/v1/auth/reset-password)
- Email verification workflow
- OAuth2 integration (Google, GitHub)

3.2 Products Module

Status: 80% Complete

Core CRUD operations are functional with pagination and basic filtering.

Working Features:

- Complete CRUD operations (Create, Read, Update, Delete)
- Product listing with pagination and status filtering
- Product specifications model (nut count, moisture content, etc.)

- Seller-only product creation and modification
- Sample data with 6 cashew products

Available Endpoints:

- POST /api/v1/products - Create product (seller only)
- GET /api/v1/products - List products (with pagination, status filter)
- GET /api/v1/products/{id} - Get product details
- PUT /api/v1/products/{id} - Update product
- DELETE /api/v1/products/{id} - Delete product

Pending Implementation:

- Advanced search endpoint (country, certifications, price range)
- Image upload endpoint for product photos (S3/Cloudinary)
- Bulk product import (CSV/Excel)
- Product analytics (view count, favorites)

Implementation Reference - Advanced Search:

```
# Add to app/products/router.py @router.get("/search",
response_model=List[schemas.ProductResponse]) def search_products( country:
Optional[str] = None, min_price: Optional[float] = None, max_price:
Optional[float] = None, certifications: Optional[str] = None, db: Session =
Depends(get_db) ): return service.search_products(db, country, min_price,
max_price, certifications)
```

3.3 Orders Module

Status: 70% Complete

Order creation, tracking, and status management are functional.

Working Features:

- Order creation workflow
- Order listing with role-based filtering
- Order details retrieval
- Order status updates
- Order timeline tracking with event history
- Activity logging integration

- Database models: orders, order_items, order_timeline

Available Endpoints:

- POST /api/v1/orders - Create new order
- GET /api/v1/orders - List orders (role-filtered)
- GET /api/v1/orders/{id} - Get order details
- PUT /api/v1/orders/{id}/status - Update order status

Pending Implementation:

- Payment tracking endpoints
- Order cancellation workflow (POST /api/v1/orders/{id}/cancel)
- Bulk order operations
- Order analytics and reporting

3.4 Quality Control Module

Status: 60% Complete

QC inspection workflow is operational from creation to approval/rejection.

Working Features:

- QC inspection creation and management
- Inspection listing and retrieval
- QC report creation
- Inspection approval workflow
- Inspection rejection workflow
- Database models: qc_inspections, qc_reports

Available Endpoints:

- POST /api/v1/qc/inspections - Create inspection
- GET /api/v1/qc/inspections - List inspections
- GET /api/v1/qc/inspections/{id} - Get inspection details
- POST /api/v1/qc/reports - Create QC report
- POST /api/v1/qc/inspections/{id}/approve - Approve inspection
- POST /api/v1/qc/inspections/{id}/reject - Reject inspection

Pending Implementation:

- Inspection photo upload (S3/Cloudinary integration)
- PDF report generation (reportlab/weasyprint)
- QC analytics dashboard
- Inspection templates

3.5 AI Integration Module

Status: 0% - Not Started

The AI module requires implementation of 5 endpoints using OpenRouter API with Claude 3.5 Sonnet model. API key is already configured in environment variables.

Required Endpoints:

• **POST /api/v1/ai/generate-description** - Generate product descriptions from basic details (product_name, category, origin, features)

• **POST /api/v1/ai/predict-price** - Analyze market prices and provide predictions (product_type, quantity, quality, origin)

• **POST /api/v1/ai/analyze-quality** - Analyze product quality from uploaded images (JPEG/PNG input)

• **POST /api/v1/ai/review-contract** - Extract and analyze contract terms (text or PDF input)

• **POST /api/v1/ai/market-insights** - Provide market trends and insights (product, region, time period)

Technical Specifications:

• AI Provider: OpenRouter (https://openrouter.ai/api/v1)

• Model: anthropic/claude-3.5-sonnet

• API Key: Pre-configured in OPENROUTER_API_KEY environment variable

• Rate Limiting: 20 calls/minute (global), 200 calls/day per user

• Error Handling: Implement retry logic and fallback responses

• Cost Tracking: Log usage for each endpoint call

Implementation Notes:

Create app/ai/ directory with router.py, schemas.py, and service.py. The service layer should handle OpenRouter API calls with proper error handling. Each endpoint should validate inputs using Pydantic schemas and return structured responses.

3.6 Admin Module

Status: 30% Complete

Basic structure exists. Dashboard analytics and user management endpoints need implementation.

Pending Endpoints:

• GET /api/v1/admin/dashboard - Dashboard analytics (total users, orders, products, revenue)

• GET /api/v1/admin/users - List all users with filters

• PUT /api/v1/admin/users/{id}/status - Activate/deactivate users

• PUT /api/v1/admin/users/{id}/role - Update user roles

- GET /api/v1/admin/orders/stats - Order statistics and reports
- GET /api/v1/admin/system/health - System health monitoring

3.7 Notifications Module

Status: 0% - Not Started

Email notification system needs SMTP configuration and background task processing.

Required Features:

- SMTP integration (Gmail, SendGrid, or AWS SES)
- Email templates for order confirmations, status updates, QC results
- Background task processing using FastAPI BackgroundTasks
- Notification preferences per user
- Optional: SMS notifications (Twilio)
- Optional: WebSocket for real-time notifications

4. Pending Implementation Tasks

High Priority Tasks

Task	Module	Description
AI Endpoints	AI Integration	Implement 5 AI endpoints using OpenRouter
Rate Limiting	Infrastructure	Configure Redis and implement rate limiting middleware
Input Sanitization	Security	Add input sanitization across all endpoints
Image Upload	Products/QC	S3 or Cloudinary integration for file uploads
Email Notifications	Notifications	SMTP setup and email template system
Advanced Search	Products	Implement multi-filter search endpoint
Payment Tracking	Orders	Payment status endpoints and tracking

5. Technical Architecture

Architecture Overview

Modular monolith architecture using FastAPI with clear separation of concerns. Each module follows the same structure: router.py (endpoints), models.py (database models), schemas.py (Pydantic validation), and service.py (business logic).

Technology Stack:

- **Backend Framework:** FastAPI (Python) with async support
- **Database:** PostgreSQL via NeonDB with SSL connection
- **Authentication:** JWT tokens with bcrypt password hashing
- **ORM:** SQLAlchemy with Alembic for migrations
- **AI Integration:** OpenRouter API (Claude 3.5 Sonnet)
- **File Storage:** S3 or Cloudinary (pending implementation)

Database Configuration

PostgreSQL database hosted on NeonDB with 14 fully migrated tables. All tables have proper indexes for query optimization.

Database Tables:

- users - User accounts and authentication
- user_profiles - Extended user information
- activity_logs - System audit trail
- products - Product catalog
- product_specifications - Product technical details
- orders - Order management
- order_items - Order line items
- order_timeline - Order event history
- qc_inspections - Quality control inspections
- qc_reports - QC inspection reports
- countries - Country reference data
- offers - Promotional offers

- featured_suppliers - Featured vendors
- system_config - System configuration

Database Connection:

```
Host: ep-quiet-bird-a122rekh-pooler.ap-southeast-1.aws.neon.tech Database: neondb SSL Mode: Required Connection String: Available in DATABASE_URL environment variable
```

Database Metrics:

Metric	Value
Sample Products	6 products
Countries Data	150+ countries
Average Query Time	< 50ms
Migration Status	All migrations applied
Index Coverage	All primary queries indexed

6. Security & Infrastructure Setup

Implemented Security Features

- JWT-based authentication with token expiry
- Bcrypt password hashing with salt rounds
- Role-based access control on all endpoints
- SSL/TLS database connections
- Environment-based configuration (.env file)
- API key protection (not committed to git)

Required Security Implementations

- **Redis Configuration:** Install and configure Redis for caching and session management
- **Rate Limiting:** Implement rate limiting middleware using Redis to prevent API abuse
- **Input Sanitization:** Add sanitization library (bleach) for all user inputs to prevent XSS
- **CORS Configuration:** Update CORS settings with production domain whitelist (remove localhost after testing)
- **API Key Rotation:** Implement API key rotation mechanism for OpenRouter
- **Request Validation:** Strengthen Pydantic schemas with strict validation rules

Redis Setup Instructions

```
1. Install Redis: sudo apt update && sudo apt install redis-server 2. Start Redis: sudo systemctl start redis sudo systemctl enable redis 3. Add to .env file: REDIS_URL=redis://localhost:6379 4. Install Python Redis client: pip install redis --break-system-packages 5. Implement rate limiting middleware in app/core/middleware.py
```

Infrastructure Components

- FastAPI application server running on port 8000
- PostgreSQL database (NeonDB cloud-hosted)
- Redis (requires installation and configuration)
- SMTP server for email notifications (pending configuration)
- S3 or Cloudinary for file storage (pending integration)
- Nginx reverse proxy (for production deployment)

- SSL/TLS certificates (Let's Encrypt for production)

7. Production Deployment Configuration

Server Requirements

- **Minimum Specifications:** 2 CPU cores, 4GB RAM, 50GB SSD storage, Ubuntu 22.04 LTS
- **Recommended Specifications:** 4 CPU cores, 8GB RAM, 100GB SSD storage
- **Network:** 100 Mbps minimum bandwidth
- **Operating System:** Ubuntu 22.04 LTS or similar Linux distribution

Environment Variables

Required environment configuration (.env file):

```
# Database Configuration
DATABASE_URL=postgresql://user:pass@host.neon.tech:5432/neondb?sslmode=require # Security JWT_SECRET_KEY=your-secret-key-minimum-32-characters
ACCESS_TOKEN_EXPIRE_MINUTES=60 # AI Integration (OpenRouter)
OPENROUTER_API_KEY=sk-or-v1-xxxxxxxxxxxxx
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
OPENROUTER_DEFAULT_MODEL=anthropic/claude-3.5-sonnet # Email Configuration (SMTP) SMTP_HOST=smtp.gmail.com SMTP_PORT=587 SMTP_USER=your_email@gmail.com SMTP_PASSWORD=your_app_password # Redis Configuration
REDIS_URL=redis://localhost:6379 # Frontend URLs (CORS)
FRONTEND_URL_BUYER=http://localhost:3001
FRONTEND_URL_SELLER=http://localhost:3000
FRONTEND_URL_AGENT=http://localhost:3002
FRONTEND_URL_ADMIN=http://localhost:3003 # File Storage (S3 or Cloudinary)
AWS_ACCESS_KEY_ID=your_access_key AWS_SECRET_ACCESS_KEY=your_secret_key
AWS_S3_BUCKET=ompoi-uploads AWS_REGION=ap-southeast-1
```

Deployment Steps

1. Server Setup: Provision VPS, update system packages, install Docker and Docker Compose

2. Code Deployment: Clone repository, create production .env file with all required variables

3. Database Migration: Run Alembic migrations: alembic upgrade head

4. Application Start: Build and start containers: docker-compose up -d

5. Nginx Configuration: Configure reverse proxy to port 8000, setup SSL with Let's Encrypt

6. Monitoring: Setup application logging, configure error tracking (Sentry optional), add health check endpoints

Testing Endpoints

Sample API test commands:

```
# Test User Registration curl -X POST http://localhost:8000/api/v1/auth/register \ -H "Content-Type: application/json" \ -d '{ "email": "developer@ompoi.com", "password": "SecurePass123!", "role": "buyer", "company_name": "Test Company" }' # Test Login curl -X POST http://localhost:8000/api/v1/auth/login \ -H "Content-Type: application/x-www-form-urlencoded" \ -d "username=developer@ompoi.com&password;=SecurePass123!" # Test Product Listing curl -X GET http://localhost:8000/api/v1/products \ -H "Authorization: Bearer YOUR_TOKEN_HERE" # Health Check curl -X GET http://localhost:8000/health
```

This developer guide is maintained by Sudhir Giri. For technical questions or implementation support, refer to the module-specific code comments and API documentation at /docs when the server is running.